Computer Networks Lab
Fall 2025
Week 6

Socket Programming (UDP)

Socket

A socket is one endpoint of a two-way communication link between two programs running on a network. Just like in TCP, sockets in UDP are also used for inter-process communication (IPC). However, UDP sockets are **connectionless**, meaning there is no need to establish or accept a connection before sending data.

In UDP, communication takes place using *datagrams*. Each message is sent independently, and the delivery, order, or duplication of packets is not guaranteed. UDP is faster but less reliable compared to TCP.

UDP vs TCP

- **TCP (Transmission Control Protocol)** → Reliable, connection-oriented, error-checked, ordered.
- **UDP (User Datagram Protocol)** → Fast, connectionless, no delivery guarantee, messages can arrive out of order or get lost.

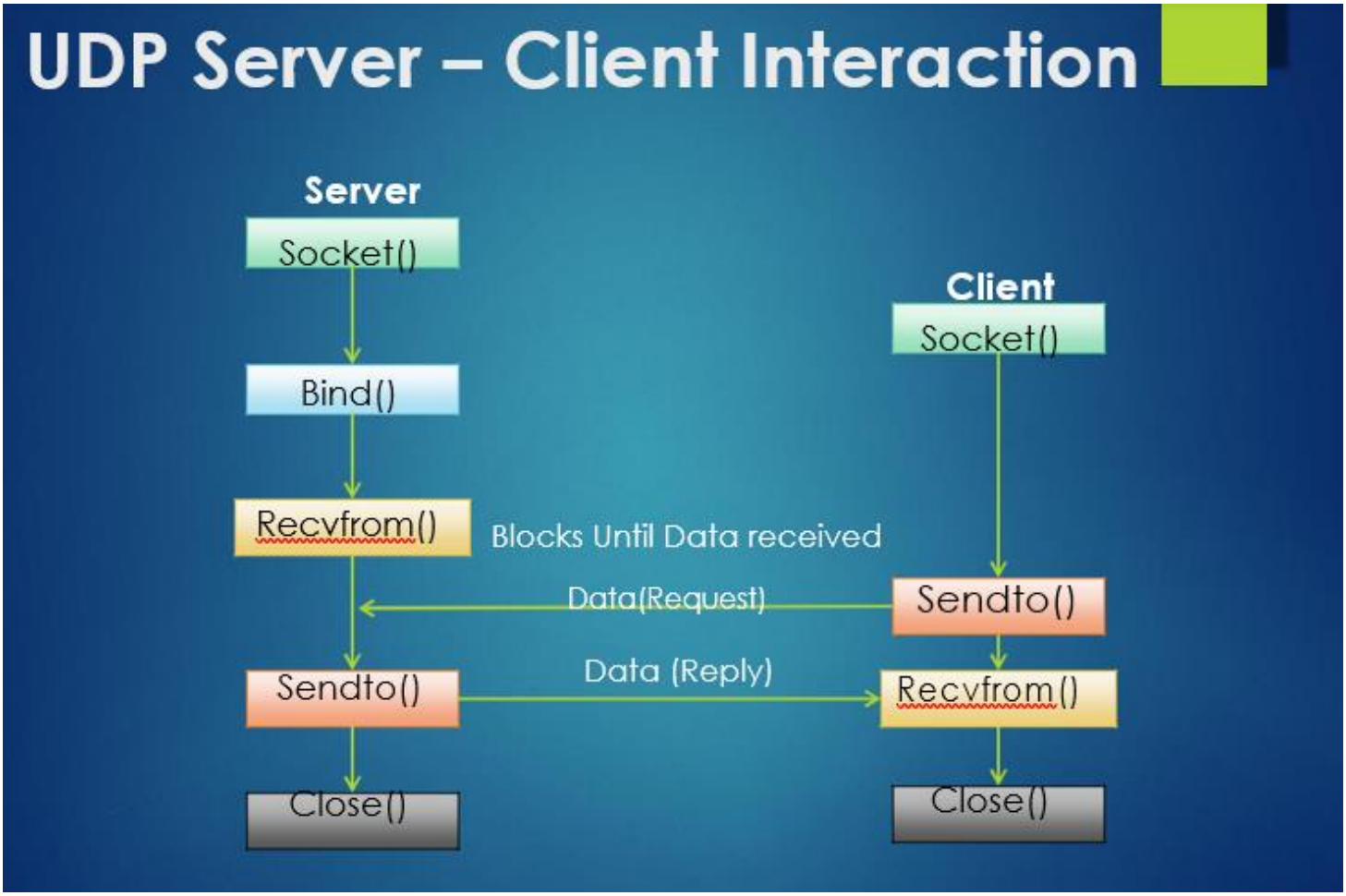
Typical UDP uses: video streaming, VoIP, DNS queries, online gaming.

Socket Function Calls

- **socket():** To create a socket
- **bind():** Assign a local IP and port to the socket
- **sendto():** Send data to a specific address
- **recvfrom():** Receive data from a specific address
- **close():** Close the socket

Unlike TCP, UDP does **not** use listen() and accept(). connect() is optional for UDP — when used it sets a default peer but does not establish a reliable connection.

Socket Programming (UDP)



Stages for Server

The server is created using the following steps:

1. Socket Creation

int sockfd = socket(domain, type, protocol)

sockfd: socket descriptor, an integer (like a file handle)
domain: integer, specifies communication domain.
type: communication type

SOCK_STREAM: TCP(reliable, connection-oriented)
SOCK_DGRAM: UDP(unreliable, connectionless)

protocol: Protocol value for Internet Protocol(IP), which is 0. This is the same number that appears on the protocol field in the IP header of a packet. It is usually set to 0 so the system automatically chooses the correct protocol (IPPROTO_UDP for UDP, IPPROTO_TCP for TCP).

```
// create socket
int sockfd;
sockfd = socket(AF_INET, SOCK_DGRAM, 0);
```

2. Bind

*int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);*

After the creation of the socket, the bind function binds the socket to the address and port number specified in addr(custom data structure). In the example code, we bind the server to the localhost, hence we use

INADDR_ANY to specify the IP address.

```
struct sockaddr_in server_addr, client_addr;
socklen_t addr_size = sizeof(client_addr);
// server address setup
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);
server_addr.sin_addr.s_addr = INADDR_ANY;

// bind socket
bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr));
```

3. Receive and send messages

*ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,*
*struct sockaddr *src_addr, socklen_t *addrlen);*

- **sockfd** → The socket descriptor that identifies the UDP socket.
- **buf** → The buffer (array) where the received data will be stored.
- **len** → The maximum number of bytes to receive.
- **flags** → Special options that modify the behavior of receiving (normally set to 0).
- **src_addr** → A structure that will hold the address of the sender.
- **addrlen** → A pointer that stores the size of the sender's address.

Receives a datagram from a UDP socket and stores the sender's address (so that the server knows who sent the message).

*ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,*
*const struct sockaddr *dest_addr, socklen_t addrlen);*

- **sockfd** → The socket descriptor that identifies the UDP socket.
- **buf** → The message (data) that will be sent.
- **len** → The size of the message in bytes.
- **flags** → Special options that modify the behavior of sending (normally set to 0).
- **dest_addr** → A structure that holds the address of the receiver.
- **addrlen** → The size of the destination address.

Sends a datagram to a specific destination (server or client) without establishing a connection.

```
// receive message from client
int n = recvfrom(sockfd, buffer, sizeof(buffer)-1, 0,
                 (struct sockaddr*)&client_addr, &addr_size);
buffer[n] = '\0'; // null-terminate
printf("Message from client: %s\n", buffer);
```

```
// send reply
sendto(sockfd, message, strlen(message)+1, 0,
       (struct sockaddr*)&client_addr, addr_size);
```

Stages for Client

1. Socket connection

Exactly the same as that of server's socket creation

```
// create socket
int sockfd;
sockfd = socket(AF_INET, SOCK_DGRAM, 0);
```

2. Client address setup:

```
struct sockaddr_in server_addr;
socklen_t addr_size = sizeof(server_addr);
// server address setup
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);
server_addr.sin_addr.s_addr = INADDR_ANY;
```

3. Send and receive message:

Send Message (Client Side)

*ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,*
*const struct sockaddr *dest_addr, socklen_t addrlen);*

- **sockfd** → The socket descriptor that identifies the client's UDP socket.
- **buf** → The message that the client wants to send.
- **len** → The size of the message in bytes.
- **flags** → Special options that modify how data is sent (normally 0).
- **dest_addr** → A structure containing the server's IP address and port number.
- **addrlen** → The size of the server's address structure.

Sends a datagram (message) from the client to the server's IP and port without creating a dedicated connection.

```
// send message
sendto(sockfd, request, strlen(request)+1, 0,
    (struct sockaddr*)&server_addr, addr_size);
```

Receive Reply (Client Side)

*ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,*
*struct sockaddr *src_addr, socklen_t *addrlen);*

- **sockfd** → The socket descriptor that identifies the client's UDP socket.
- **buf** → The buffer where the reply from the server will be stored.
- **len** → The maximum number of bytes the client can receive.
- **flags** → Special options that modify how data is received (normally 0).
- **src_addr** → A structure that will store the server's IP address and port number.
- **addrlen** → A pointer to the size of the server's address structure.

Receives a datagram reply from the server, along with the server's address information.

```
// receive reply
int n = recvfrom(sockfd, buffer, sizeof(buffer)-1, 0,
|   |   |   |   |(struct sockaddr*)&server_addr, &addr_size);
buffer[n] = '\0'; // null-terminate
printf("Message from server: %s\n", buffer);
```

Sample Code:

Server Side

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>


#define PORT 9000

int main() {
    char buffer[256];
    char message[256] = "Hello from UDP server!";
    int sockfd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size = sizeof(client_addr);

    // create socket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    // setup server address
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = INADDR_ANY;

    // bind socket
    bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr));

    // receive message from client
    int n = recvfrom(sockfd, buffer, sizeof(buffer)-1, 0,
                     (struct sockaddr*)&client_addr, &addr_size);
    buffer[n] = '\0'; // null-terminate
    printf("Message from client: %s\n", buffer);

    // send reply
    sendto(sockfd, message, strlen(message)+1, 0,
           (struct sockaddr*)&client_addr, addr_size);

    close(sockfd);
    return 0;
}
```

Client Side

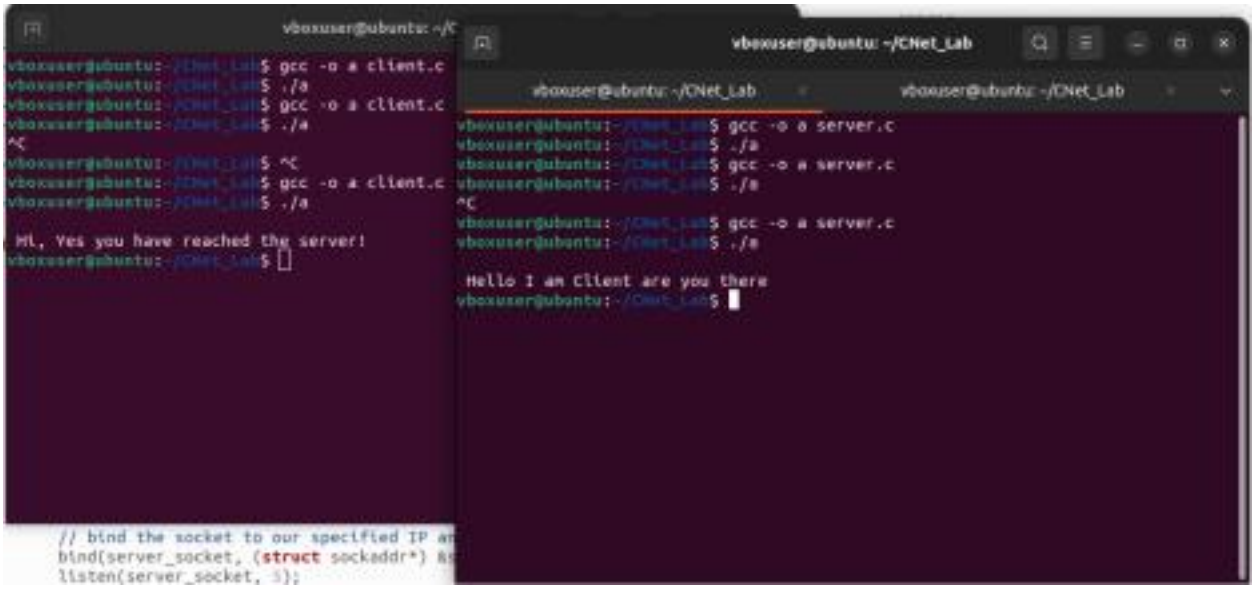
```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>


#define PORT 9000

int main() {
    char buffer[256];
    char request[256] = "Hello UDP server, how are you?";
    int sockfd;


    // create socket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    struct sockaddr_in server_addr;
    socklen_t addr_size = sizeof(server_addr);
    // server address setup
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = INADDR_ANY;

    // send message
    sendto(sockfd, request, strlen(request)+1, 0,
           (struct sockaddr*)&server_addr, addr_size);

    // receive reply
    recvfrom(sockfd, buffer, sizeof(buffer), 0,
             (struct sockaddr*)&server_addr, &addr_size);
    printf("Message from server: %s\n", buffer);

    close(sockfd);
    return 0;
}
```

How to Run the code?

Open two separate terminals, each accessing the folder you have your code in. Compile both the files using the command : ***gcc filename.c -o exefilenameTobeassigned***

Once compiled without errors, access the server exe file, and then the client exe file.

For reference see the image attached below.


vboxuser@ubuntu: ~/CNet_Lab
vboxuser@ubuntu: ~/CNet_Lab
vboxuser@ubuntu: ~/CNet_Lab
vboxuser@ubuntu: ~/CNet_Lab
$ gcc -o a client.c
$ ./a
$ gcc -o a client.c
$ ./a
^C
$ ^C
$ gcc -o a client.c
$ ./a
Hi, Yes you have reached the server!
$ gcc -o a server.c
$ ./a
$ gcc -o a server.c
$ ./a
^C
$ gcc -o a server.c
$ ./a
Hello I am Client are you there
// bind the socket to our specified IP a
bind(server_socket, (struct sockaddr*) &
listen(server_socket, 5);